

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**
2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. **(5 marks)**
3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. **(5 marks)**
4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. **(5 marks)**

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance. **(5 marks)**

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. **(5 marks)**

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. **(5 marks)**

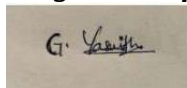
9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. **(5 marks)**

10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. **(5 marks)**

Code of Conduct:

*I **G.Yaswitha(192325087)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



1. RL Framework (Agent–Environment Interaction)
Problem Statement

Understand how an RL agent interacts with an environment using states, actions, and rewards, and how cumulative reward drives decision-making.

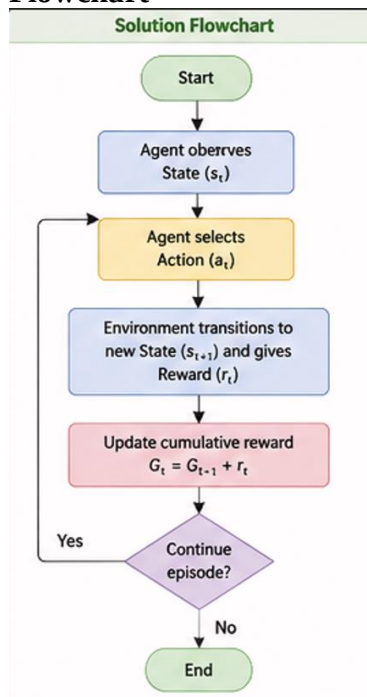
Solution Explanation

- **Agent:** Learner/decision maker
- **Environment:** External system
- **State (S):** Current situation
- **Action (A):** Decision taken
- **Reward (R):** Feedback signal

The agent:

1. Observes state
2. Takes action
3. Receives reward
4. Moves to next state

Flowchart



Code (Simple Simulation)

```
import random

state = 0
total_reward = 0

for step in range(5):
    action = random.choice(["left", "right"])
    reward = random.randint(-1, 5)
    total_reward += reward
    print("Step:", step, "Action:", action, "Reward:", reward)

print("Total Reward:", total_reward)
```

Sample Output

```
Step: 0 Action: left Reward: 2
Step: 1 Action: right Reward: 1
Step: 2 Action: left Reward: 3
```

Step: 3 Action: right Reward: -1

Step: 4 Action: left Reward: 4

Total Reward: 9

2. Markov Decision Process (MDP)

Problem Statement

Model decision-making problems using MDP with components like states, actions, transition probability, reward function, discount factor.

Solution Explanation

MDP defined as:

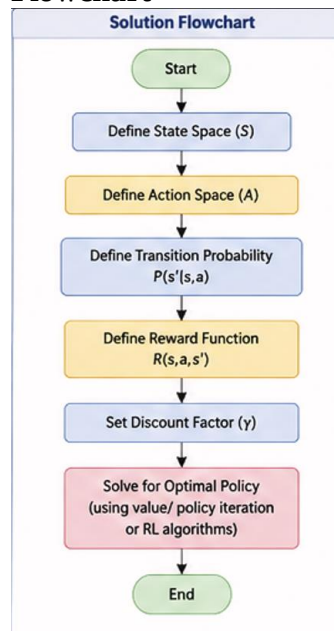
[
(S, A, P, R, γ)
]

- **S**: States
- **A**: Actions
- **P(s'|s,a)**: Transition probability
- **R(s,a)**: Reward
- **γ (gamma)**: Discount factor

Markov Property:

Future depends only on current state

Flowchart



Code

```
states = ["S1", "S2"]  
actions = ["A1", "A2"]
```

```
transition = {  
    ("S1", "A1"):(("S2", 1),  
    ("S2", "A2"):(("S1", 2)  
}
```

```
gamma = 0.9
```

```
for s,a in transition:
    next_s, reward = transition[(s,a)]
    value = reward + gamma*0
    print(s,a,"->",next_s,"Reward:",reward)
```

Sample Output

```
S1 A1 -> S2 Reward: 1
S2 A2 -> S1 Reward: 2
```

3. Policy & Value Functions (Bellman Equation)

Problem Statement

Explain how policy and value functions help evaluate long-term rewards using the Bellman Equation.

Solution Explanation

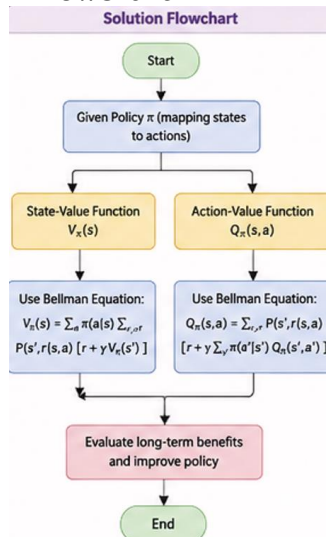
- **Policy (π):** Strategy (mapping states $\rightarrow$ actions)
- **State Value ($V(s)$):** Value of being in state
- **Action Value ($Q(s,a)$):** Value of action in state

Bellman Equation:

$$V(s) = R + \gamma V(s')$$

$$Q(s,a) = R + \gamma \max_{a'} Q(s',a')$$

Flowchart



Code

```
gamma = 0.9
reward = 5
next_value = 10
```

```
value = reward + gamma * next_value
```

```
print("Updated Value:", value)
```

Sample Output

Updated Value: 14.0

4. Exploration vs Exploitation

Problem Statement

Justify exploration–exploitation trade-off and compare ϵ -greedy and softmax strategies.

Solution Explanation

- **Exploration:** Try new actions
- **Exploitation:** Use best known action

Strategies:

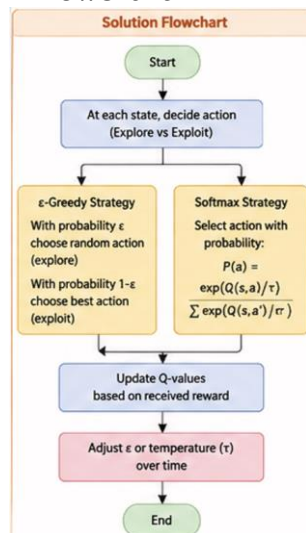
1. ϵ -greedy

- With probability $\epsilon \rightarrow$ random action
- Else $\rightarrow$ best action

2. Softmax

- Probabilistic selection based on value
- Higher value $\rightarrow$ higher probability

Flowchart



Code

```
import random
```

```
epsilon = 0.3
```

```
actions = ["A", "B", "C"]
```

```
best_action = "A"
```

```
for i in range(5):
```

```
    if random.random() < epsilon:
```

```
        action = random.choice(actions)
```

```
        print("Exploration:", action)
```

else:

```
    print("Exploitation:", best_action)
```

Sample Output

Exploration: B

Exploitation: A

Exploitation: A

Exploration: C

Exploitation: A

5. Reward Shaping

Problem Statement

Explain how reward shaping improves learning speed and prevents suboptimal policies.

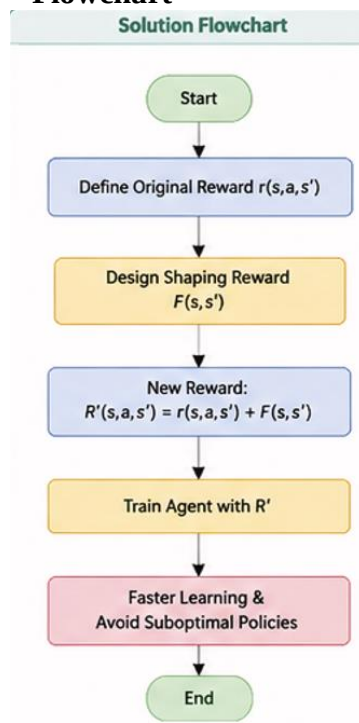
Solution Explanation

- Adds extra guidance rewards
- Helps agent learn faster
- Avoids getting stuck in poor policies

Example:

- Without shaping → sparse rewards
- With shaping → intermediate rewards

Flowchart



Code

```
goal_state = 10
```

```
state = 0
```

```
for step in range(5):
```

```
    state += 2
```

```
    reward = -1
```

```

if state == goal_state:
    reward += 10
else:
    reward += 1 # shaping reward
print("State:",state,"Reward:",reward)

```

Sample Output

State: 2 Reward: 0
 State: 4 Reward: 0
 State: 6 Reward: 0
 State: 8 Reward: 0
 State: 10 Reward: 10

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately